

Evidence-Backed Release Assurance for Autonomous Agent Deployments: Cryptographic Attestation, Fail-Closed Gates, and Tamper-Resilient Audit Infrastructure

Anonymous Submission
Submission ID: EVI-227

Abstract

As autonomous software agents execute sensitive workflows, interact with enterprise APIs, and modify persistent infrastructure, deploying model updates or altering tool permissions introduces severe systems security risks. Existing software delivery pipelines rely on unauthenticated runner logs or process exit codes, leaving a critical gap: while traditional frameworks attest static code artifacts, agents are governed by dynamic, non-deterministic execution traces. This paper presents Evidence Assurance (EVIASSURE), a cryptographic release assurance architecture for autonomous agent deployments. EVIASSURE elevates execution logs into an *evidence-bound*, *fail-closed release authorization layer*, where deployment authorization is a deterministic function of cryptographically attested trace evidence evaluated under pinned governance policies. The architecture introduces: (1) a Merkle-based trace attestation engine binding action histories with depth-enforced tree proofs; (2) an evidence envelope supporting multi-signature governance and privacy-preserving salted parameter blinding; (3) a zero-trust policy engine enforcing freshness, key revocation, and replay protection; and (4) a two-layer forensics model separating cryptographic integrity verification from semantic trace inspection. In empirical benchmarks, EVIASSURE achieves a 100.0% fail-closed block rate across 13 adversarial attack vectors and a 1,000-payload mutation fuzzing campaign (compared to 0.0% for exit-code gates, 23.1% for schema gates, and 30.8% for a composed Sigstore+OPA baseline) while sustaining 7,152 verification operations per second.

1 Introduction

Autonomous software agents driven by Large Language Models (LLMs) operate dynamically across complex tool environments, API endpoints, and persistent memory stores [10, 32, 33]. Unlike static software artifacts whose behaviors are deterministically governed by compiled

code, autonomous agents exhibit non-deterministic reasoning patterns, dynamic tool execution flows, and adaptive multi-step decision sequences. When deploying agent updates, expanding tool privilege scopes, or promoting agentic pipelines to production, organization-wide security depends upon verifying that safety invariants, authorization constraints, and regression test suites have been rigorously validated [10].

Existing software delivery and release pipelines suffer from a fundamental trust defect: traditional deployment gates evaluate unauthenticated status signals (such as process exit codes, plain text console logs, or un-signed JavaScript Object Notation (JSON) build status files) generated by intermediate runner environments. An attacker who gains control over an intermediate build step, manipulates agent memory traces, or exploits prompt injection vectors can easily forge release approval signals, bypassing security verification entirely [7, 24, 29, 31].

Empirical studies of the open-source ecosystem confirm both the prevalence and the practical exploitability of these compromise paths: taxonomies of documented supply-chain attacks now enumerate hundreds of real incidents [16, 22], large-scale measurement work shows package-manager compromise to be routine in interpreted-language ecosystems [9], and multi-repository update systems require dedicated defenses [20].

This leaves an exploitable gap: existing supply chain security focuses almost entirely on attesting the origin and integrity of static software artifacts (e.g., container images, compiled binaries). However, the security posture of an autonomous agent is not defined by its code artifact alone, but by its dynamic, non-deterministic execution traces against a test suite. The fundamental problem is that while we can attest *what* an agent is, current systems cannot cryptographically authorize a release based on *how it behaved*.

To address these critical vulnerabilities, EviAssure¹

¹The complete anonymous source code, evaluation corpus, and benchmark suite are available at <https://anonymous.4open>.

introduces an evidence-backed release assurance architecture that fills this niche. It replaces informal release signals with cryptographically authenticated Evidence Bundles backed by Merkle inclusion proofs, timestamp freshness windows, digital signatures, and provenance envelopes.

1.1 Key Contributions

The primary technical and systems contributions of this work are:

1. **Evidence-Bound Release Authorization:** The system-level contribution is an authorization layer, not just a record format. EviAssure outputs a deterministic, reproducible verdict that is a function of attested evidence under a pinned policy, meaning an APPROVED release carries its own security proof.
2. **Cryptographic Trace Attestation & Blinding:** A Merkle tree aggregation protocol that condenses agent execution traces into an immutable root digest with signed trace counts for completeness-carrying binding, alongside salted parameter blinding to preserve public auditability while protecting privacy.
3. **Formal Soundness & Two-Layer Forensics Model:** A system-level *Release Authorization Soundness* theorem formalizing the distinction between cryptographic evidence integrity and trace completeness, paired with a two-layer model separating gate-level integrity from per-leaf semantic inspection for robust forensics and diagnostics.
4. **Comprehensive Empirical Evaluation & Corpus:** Benchmarks demonstrating a 100.0% fail-closed block rate across 13 attack vectors and high-throughput policy evaluations, evaluated using a 1,050-profile multi-architecture agent trace corpus with planted anomalies.

Record Formats versus Authorization Layers. A fundamental distinction separates EviAssure from existing software supply chain metadata standards. Frameworks like in-toto provide signed, tamper-evident attestations of supply-chain steps—including vetted runtime-trace predicates [12]—and Supply chain Levels for Software Artifacts (SLSA) builds graduated release provenance on top [29, 31]. However, these frameworks define *passive record formats*. They attest *what occurred* during a static build, but lack a *stateful authorization layer* to actively gate a release based on non-deterministic execution traces. EviAssure is an *active, fail-closed authorization gate*. EviAssure’s `EvidenceBundle` functions as an in-toto v0.2 / SLSA v1.0 compatible envelope, composing directly with existing metadata specifications.

science/r/eviassure-release-assurance/.

EviAssure provides the enforcement mechanisms these specifications omit: (i) completeness-carrying trace binding via a signed trace count, (ii) stateful freshness and replay protection enforced integrally at the gate, and (iii) a mathematically reproducible release decision under a pinned policy. Section 8.5 compares EviAssure against recent 2026 AI-agent provenance standards in detail.

Open Science and Reproducibility. To support open science and ensure complete reproducibility of our claims, the full source code of EviAssure, the 1,050-profile execution trace corpus, the 13-vector attack harness, and the baseline evaluation datasets (including the precise Open Policy Agent (OPA) Rego policies and unauthenticated JSON payloads used to establish the baseline in Section 7.8) are released as a sanitized, open-source artifact via an Anonymous GitHub repository (see Appendix B).

2 Preliminaries

2.1 Formal Security Definitions

Definition 1 (Unforgeability under Chosen Message Attack (EUF-CMA)). *An Evidence Assurance Scheme $\Sigma = (\text{KeyGen}, \text{Package}, \text{Verify})$ is EUF-CMA secure if for all probabilistic polynomial-time adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ outputs a valid pair (E^*, σ^*) such that $\text{Verify}(PK, E^*, \sigma^*) = 1$ without E^* having been signed by an honest party holding SK is negligible in security parameter λ :*

$$\Pr[\text{Exp}_{\Sigma, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1] \leq \text{negl}(\lambda) \quad (1)$$

Definition 2 (Merkle Trace Inclusion Soundness). *For root R_M committed together with its leaf count N , and leaf H_i , a Merkle proof π_i is sound if no computationally bounded adversary can construct π'_i for any $H'_i \neq H_i$ — including internal node digests of the same tree — such that $\text{VerifyMerkle}(H'_i, \pi'_i, R_M, \lceil \log_2 N \rceil) = 1$, where verification accepts only proof paths of the committed tree depth.*

Definition 3 (Salted Trace Parameter Blinding Indistinguishability). *Under the standard assumption that HMAC-SHA256 [14] is a secure Pseudorandom Function (PRF) family $\mathcal{F} : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, a blinded trace record T_{blinded} produced with uniform salt $S \leftarrow \{0, 1\}^\lambda$ satisfies computational indistinguishability: no PPT adversary $\mathcal{A}$ without knowledge of S can distinguish T_{blinded} from a uniformly random bitstring $\{0, 1\}^\lambda$ with non-negligible advantage $\text{Adv}_{\mathcal{F}}^{\text{prf}}(\mathcal{A})$.*

Definition 4 (Evidence Integrity). *An evidence pack (R_M, σ, N) comprising a Merkle root R_M , signature σ ,*

and signed trace count N provides evidence integrity if no probabilistic polynomial-time adversary can produce any $(R'_M, \sigma', N') \neq (R_M, \sigma, N)$ that verifies; equivalently, nothing that was recorded can be modified, reordered, inserted, or removed without detection. Integrity is a property of the signed record and is guaranteed by Theorem 1 below.

Definition 5 (Trace Completeness). *Let E be an agent execution and let $\mathcal{C}$ be the evidence collector that observes E . A recorded trace set $T = \{T_1, \dots, T_N\}$ is complete with respect to E if every action of E that $\mathcal{C}$ observed is represented by exactly one T_i . Completeness is an observation property of the collector, not of the Merkle tree: the tree attests exactly the leaves it was given, and no downstream check can distinguish an action that was never recorded from one that was honestly absent. For example, if a real execution sequence is $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$, but the recorded trace is $A \rightarrow B \rightarrow D \rightarrow E$, the Merkle tree provides strong integrity protection for the recorded sequence, but it cannot establish that step C was omitted. In production deployments, complete mediation is grounded at the OS/hypervisor boundary via kernel-level system call tracing (e.g., eBPF or seccomp filters), ensuring that no unmediated side-effects escape the agent runner without reaching $\mathcal{C}$ prior to tree construction.*

3 System Architecture and Threat Model

Intuition and Terminology. The core intuition of the EviAssure scheme is to transform the non-deterministic actions of autonomous agents into a cryptographically verifiable ledger prior to deployment. Rather than trusting the agent’s self-reported success or scanning final artifacts, EviAssure packages the agent’s exact step-by-step execution path into an immutable “evidence bundle,” hashes it into a Merkle tree, and securely signs it via an external Hardware Security Module (HSM). This bundle is then evaluated by a *fail-closed gate*—a strict policy enforcement engine that defaults to denying the release if any required cryptographic proof, signature, or policy threshold is missing, malformed, or compromised.

3.1 System Model

The EviAssure release assurance architecture involves four primary entities: the *Evidence Collector*, the *Key Management System (KMS)*, the *Policy Registry*, and the *Release Assurance Gate* (Figure 1).

Evidence Collector: In production deployments, the collector operates in an isolated out-of-process monitor container or host cgroup outside the agent’s sandboxed

execution runtime, observing agent activity via kernel-level eBPF syscall tracepoints and container network proxies without sharing memory or signal access. In our experimental evaluation testbed (Section 7), the collector is implemented via deterministic Python application-level instrumentation wrappers to ensure reproducible cross-platform benchmarking across 1,050 agent profiles. *Cloud KMS / Hardware Enclave:* To defend against local runner compromise, EviAssure roots signing authority within a Hardware Security Module (HSM) or Cloud KMS boundary. The collector requests attestation signatures over canonical evidence digests without exposing private key material. The gate enforces the Amazon Resource Name (ARN) boundary (`kms_key_arn_pattern`) at verification time regardless of where signing occurred. *Policy Registry:* A trusted, out-of-band registry storing declarative release policies and the Key Revocation List (CRL). It governs which signatures and trace properties are acceptable. *Release Assurance Gate:* The central enforcement entity. It runs in a trusted, isolated deployment environment (e.g., a secured CI/CD pipeline step) and evaluates the cryptographically signed `EvidenceBundle` against the pinned policy from the registry.

3.2 Adversary Model

The security model considers a powerful adversary $\mathcal{A}$ attempting to deploy an unapproved, regression-prone, or malicious agent update to production environments [8, 26, 27]. The adversary $\mathcal{A}$ may control intermediate runner environments, network transit layers, or build scripts.

Trust Model Boundaries. EviAssure assumes that the underlying cryptographic primitives (Ed25519 and SHA-256) remain computationally intractable for PPT adversaries, and that the Cloud KMS / HSM service faithfully enforces identity access control over private signing keys. In addition, the system requires strict environment isolation: if an autonomous agent escapes its sandboxed execution tier to gain local file access or arbitrary code execution over the assurance engine or the collector host process, it can forge an evidence payload *prior* to KMS signing. Such a compromise of the assurance engine’s trusted computing base is an explicit out-of-scope limitation, meaning the cryptographic guarantees hold assuming a tamper-proof assurance control plane. Physical hardware side-channel attacks on HSM microchips, total subversion of the cloud provider’s root KMS master keys, and hypervisor rootkits are likewise out-of-scope for the release gate layer.

We categorize $\mathcal{A}$ ’s in-scope capabilities into five adversary capability classes, which are mapped to specific mitigation mechanisms in Table 3. These cover runner and memory tampering ($\mathcal{A}_1$), network and signature

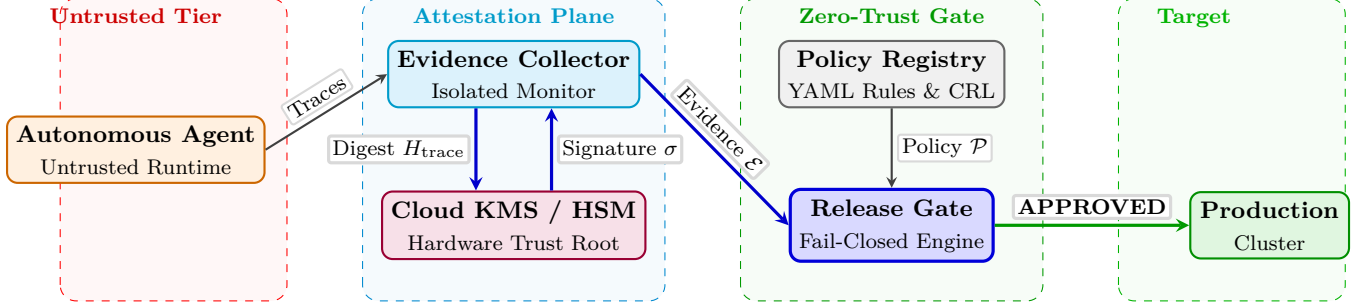


Figure 1: EviAssure System Architecture and Trust Boundaries. The Evidence Collector captures execution traces across an out-of-process monitor boundary and requests hardware-rooted attestation from Cloud KMS. The Release Assurance Gate evaluates the signed bundle ($\mathcal{E}$) against pinned governance rules ($\mathcal{P}$) from the Registry to authorize production release.

forgery ($\mathcal{A}_2$), replay and time skew ($\mathcal{A}_3$), governance bypass ($\mathcal{A}_4$), and serialization malleability ($\mathcal{A}_5$).

3.3 Security Requirements

The EviAssure construction is proved against the following fundamental security requirements, subject to the defined trust boundaries:

1. *Evidence Integrity*: The gate must ensure that no recorded action can be modified, reordered, inserted, or removed without detection (Definition 4).
2. *Trace Completeness*: While integrity is cryptographic, the system requires the collection boundary to capture all actions honestly. The gate must bind the attestation specifically to the count of recorded actions to prevent silent truncation (Definition 5).
3. *Release Authorization Soundness*: The release decision must be a deterministic function of the signed evidence and the pinned policy. No PPT adversary should induce an APPROVED verdict for modified or malicious traces.
4. *Trace Privacy (Parameter Blinding)*: Sensitive trace parameters and intermediate data should remain completely hidden from public verifiers or auditors via indistinguishable salted HMAC blinding.

The release gate enforces a strict zero-trust fail-closed guarantee against these requirements: any evidence payload failing verification is rejected with exit code 1 (BLOCKED). As discussed under trace completeness, EviAssure scopes its guarantees to the *collection boundary*—the same boundary every attested-execution system faces [1, 13]. Residual risk is mitigated by collecting evidence at the outermost observable boundary and by semantic inspection.

4 Cryptographic Evidence Attestation

EviAssure establishes verifiable release claims by packaging agent action histories into cryptographically attested evidence bundles. This section describes the Merkle tree aggregation protocol, browser Document Object Model (DOM) attestation, trace parameter blinding, and SLSA v1.0 / in-toto predicate envelopes.

4.1 Execution Trace & Browser UI Attestation Schema

Each execution trace record T_i captures structured action metadata across autonomous agent steps. For API and backend tool calls, T_i records:

$$T_i = (\text{id}_i, \text{agent}_i, \text{action}_i, \text{status}_i, \text{duration}_i, H_{\text{out},i}) \quad (2)$$

For browser and UI agent steps, EviAssure extends attestation to capture DOM state hashes (H_{DOM}) and visual screenshot render digests (H_{screen}):

$$T_{\text{UI},i} = (\text{id}_i, \text{agent}_i, \text{action}_i, \text{url}_i, \text{elem}_i, H_{\text{DOM},i}, H_{\text{screen},i}) \quad (3)$$

H_{DOM} is computed over the serialized DOM state of the rendered page, $H_{\text{DOM},i} = \text{SHA256}(\text{DOM}_{\text{serialized}})$: the specimen serializes the rendered document deterministically before hashing, so identical page states yield identical digests while any mutation (e.g., an injected modal dialog or altered form control) changes the digest. Visual render digests $H_{\text{screen},i} = \text{SHA256}(\text{render})$ capture rendered canvas pixels under deterministic software rasterization (e.g., headless Chromium with `-disable-gpu`, `-font-render-hinting=none`, and pinned viewport dimensions on an immutable container base image to eliminate cross-driver subpixel anti-aliasing drift). Each trace record produces a canonical leaf digest $H_i = \text{SHA256}(T_i.\text{to_hash}())$. The canonical leaf serialization commits *every* schema field—identifier, agent, action, status, duration, and output digest—so modifying

any recorded field after signing, including the timing `duration_ms`, invalidates the recomputed root and is rejected by the gate.

4.2 Salted Trace Parameter Blinding Protocol

To protect proprietary prompt text, customer PII, or sensitive API parameters during public audit, EviAssure implements a keyed HMAC blinding mode (`blind_payload`). The binding assumption is stated first: evidence bundles never contain raw payload strings—the trace records store only digests (`output_hash`) and the `raw_payload` field is excluded from serialization—so secrecy rests on excluding raw text at the collector, and blinding does not (and need not) conceal data that is absent from the bundle. What blinding adds is *domain separation*: given the trace’s output digest P_i and per-deployment salt $S \leftarrow \{0,1\}^{256}$:

$$H_{\text{blinded},i} = \text{HMAC-SHA256}(S, P_i) \quad (4)$$

The blinded value retains the full 256-bit HMAC digest, preserving payload-aliasing resistance at the same scale as the underlying hash while strictly maintaining indistinguishability from random under Definition 3. Salt S is maintained as a private secret between the collector and authorized verifiers; for public audits involving discrete or low-entropy parameters (e.g., boolean flags), the collector appends a per-trace 128-bit random blinding nonce prior to HMAC evaluation to prevent dictionary pre-computation attacks. It replaces raw payload strings in leaf construction while preserving global Merkle tree inclusion verification.

4.3 Merkle Trace Aggregation and Sparse Proof Compression

To attest N execution traces without transmitting full raw logs to every gate request, EviAssure aggregates normalized trace digests into a SHA-256 Merkle tree [17, 19]. Each trace record T_i maps to leaf digest H_i .

For $N = 1,000$ traces, transmitting raw trace logs requires 222.6 KB of storage, scaling to 220,705.03 KB for $N = 1,000,000$ traces. EviAssure provides sparse Merkle proof compression, transmitting the 32-byte root R_M and $O(\log N)$ inclusion proof paths π_i for audited steps.

Given proof path $\pi_i = \{(P_1, \text{pos}_1), \dots, (P_K, \text{pos}_K)\}$, inclusion is verified recursively by setting $C_0 = H_i$ (the committed leaf digest; leaf digests are already SHA-256 strings and are *not* re-hashed at the proof boundary, matching `verify_merkle_proof`) and evaluating:

$$C_j = \begin{cases} \text{SHA256}(C_{j-1} \parallel \text{“:”} \parallel P_j), & \text{if } \text{pos}_j = \text{right} \\ \text{SHA256}(P_j \parallel \text{“:”} \parallel C_{j-1}), & \text{if } \text{pos}_j = \text{left} \end{cases} \quad (5)$$

where $\parallel$ denotes string concatenation. This explicit construction uses the `:` separator character between the strictly 64-character hex-encoded SHA-256 left and right node digests. Because the internal node payload size is rigidly constrained ($64 + 1 + 64 = 129$ bytes) and structurally separated by the delimiter, it natively thwarts length-extension and collision/malleability attacks, guaranteeing robust domain separation between leaves and internal nodes. The proof is valid if and only if $C_K = R_M$ and the path length equals the committed tree depth $\lceil \log_2 N \rceil$, derived from the signed execution trace count; the length and signed-count check prevents internal node digests from being verified as leaves (and thwarts second-preimage attacks via odd-leaf duplication padding) — a substitution that would otherwise succeed without any hash collision — reducing transmission size to < 5 KB.

```
def build_merkle_tree(traces):
    leaves = [t.to_hash() for t in traces]
    current = leaves
    levels = [leaves]
    while len(current) > 1:
        next_level = []
        for i in range(0, len(current), 2):
            left = current[i]
            right = current[i+1] if i+1 < len(current)
            else current[i]
            next_level.append(hash_sha256(f"{left}:{right}"))
        current = next_level
        levels.append(current)
    return current[0], levels
```

Listing 1: Merkle Trace Aggregation Protocol.

Listing 1 implements the recursive aggregation of agent trace hashes into the final $O(\log N)$ verifiable structure.

4.4 SLSA v1.0 and in-toto Predicate Envelopes

To integrate with cloud-native artifact registries and software supply chain attestors, EviAssure formats each generated `EvidenceBundle` into an in-toto v0.2 Statement envelope compatible with SLSA Provenance v1.0 specifications [29, 31]. Listing 2 illustrates the JSON structure exported by EviAssure.

The envelope schema binds the target release artifact (e.g., agent model weight package, code container, or configuration tarball) to the evidence attestation. The `subject` field specifies the artifact filename and its SHA-256 digest. The `predicateType` URI asserts SLSA Provenance v1.0 compliance. Within `predicate.buildDefinition`, the `buildType` asserts policy gate evaluation parameters, while `externalParameters` anchors the unique `evidence_id`, binding the external evidence pack directly to the SLSA build statement.

```
{
  "_type": "https://in-toto.io/Statement/v0.1",
  "subject": [
```

```

{
  "name": "agentic_release_artifact.tar.gz",
  "digest": { "sha256": "03126da4143384d8..." }
},
"predicateType": "https://slsa.dev/provenance/v1",
"predicate": {
  "buildDefinition": {
    "buildType": "https://usenix.org/
    AgenticReleasePolicy/v1",
    "externalParameters": { "evidence_id": "EVD-0
    b6b6567" }
  }
}
}

```

Listing 2: SLSA Provenance v1.0 Statement envelope produced by EviAssure.

Listing 2 shows the structured metadata layout that binds the artifact digest directly to the evidence package for public audit.

5 Fail-Closed Policy Verification Engine

Release authorization in EviAssure is governed by a zero-trust policy engine (`ReleasePolicyEngine`) that enforces mandatory fail-closed default rules. This section details the declarative policy schema, multi-factor evaluation protocol, key governance rules, and the system-level formal security analysis.

5.1 Declarative Policy Governance Rules

A release policy specification P defines strict quantitative safety thresholds and cryptographic verification conditions:

$$P = (P_{\min_pass}, P_{\max_drift}, P_{\min_sigs}, P_{\text{valid_window}}, \mathcal{K}_{\text{trusted}}, \mathcal{K}_{\text{rev}}, \mathcal{A}_{\text{KMS}}) \quad (6)$$

where:

- $P_{\min_pass}$: Minimum required unit and integration test pass rate (default: 100.0% (100/100)).
- $P_{\max_drift}$: Maximum allowable unresolved security drift findings (default: 0).
- $P_{\min_sigs}$: Required threshold number of valid signatures from authorized keys (default: $K = 1$).
- $P_{\text{valid_window}}$: Evidence timestamp freshness validity window $\Delta T_{\max} = 3,600$ s (1 hour) with a maximum future clock skew tolerance of $\delta_{\text{skew}} = 30$ s (assuming RFC 5905 NTP synchronization across runner nodes).
- $\mathcal{K}_{\text{trusted}}$: Trusted key registry pinning authorized signing key IDs to their Ed25519 public keys (`governance/trusted_keys.yaml`). Signature verification uses the pinned key, never a public key supplied inside a submitted bundle; unregistered

key IDs are rejected. The registry follows the key-accountability patterns of key transparency [18] and delegation hierarchies [15]: verification authority rests on a small, versioned, offline-pinnable root set rather than on keys that travel with the evidence.

- $\mathcal{K}_{\text{rev}}$: Key Revocation List (CRL) tracking compromised or deprecated key identifiers.
- $\mathcal{A}_{\text{KMS}}$: Required Cloud KMS or HSM key ARN boundary pattern matching.

Listing 3 outlines the complete zero-trust verification algorithm enforced by `ReleasePolicyEngine`.

```

# Protocol 2: Fail-Closed Policy Gate Verification
# Input: Evidence E, Policy P, Revoked keys K_rev,
#       Seen nonces N_seen
# Output: Decision D in {APPROVED, BLOCKED}
def evaluate_release_gate(E, P, K_rev, N_seen):
    if not E.signed or E.signature is None:
        return BLOCKED, ["POLICY_VIOLATION: Unsigned
        payload"]

    # 1. Signature & Key Governance Verification
    valid_sigs = 0
    for s_entry in E.signatures:
        if s_entry.key_id in K_rev or s_entry.key_id
        not in K_trusted:
            continue # revoked or unregistered key
        if s_entry.pub_key != K_trusted[s_entry.key_id
        ]:
            continue # spoofed key_id under an
            attacker key
        if s_entry.kms_arn and not match_kms_arn(
            s_entry.kms_arn, P.kms_pattern):
            continue
        if verify_signature(s_entry):
            valid_sigs += 1

    if valid_sigs < P.min_signatures:
        return BLOCKED, ["POLICY_VIOLATION:
        Insufficient signatures"]

    # 2. Merkle Root Integrity Verification
    recalculated_root, _ = build_merkle_tree(E.traces)
    if recalculated_root != E.merkle_root:
        return BLOCKED, ["POLICY_VIOLATION: Merkle
        root mismatch"]

    # 3. Quality & Security Drift Thresholds
    if E.test_pass_pct < P.min_pass_pct:
        return BLOCKED, ["POLICY_VIOLATION: Sub-
        threshold pass rate"]
    if E.unresolved_drift > P.max_drift:
        return BLOCKED, ["POLICY_VIOLATION: Unresolved
        drift"]

    # 4. Nonce Replay & Freshness Windows
    if E.nonce in N_seen:
        return BLOCKED, ["POLICY_VIOLATION: Replayed
        nonce"]
    if not is_timestamp_fresh(E.timestamp, P.
        valid_window, max_skew=30):
        return BLOCKED, ["POLICY_VIOLATION: Timestamp
        expired/skewed"]

    N_seen.add(E.nonce)
    return APPROVED, []

```

Listing 3: Fail-Closed Policy Gate Verification Protocol.

Listing 3 outlines the zero-trust policy engine logic that strictly enforces fail-closed behavior on signature or trace discrepancies.

5.2 Formal Security Analysis

The security analysis proceeds in two layers. Theorem 1 is the primitive-level argument: a sequence of games showing what no adversary can do to the cryptographic components, reducing to EUF-CMA and collision resistance. Theorem 2 is the system-level argument: it states what an APPROVED verdict *means* — the actual security objective of a release gate — and makes explicit the trust boundaries (evidence collector, key registry, nonce state) on which that meaning rests.

Theorem 1 (Fail-Closed Release Soundness). *Under the assumption that Ed25519 signatures [4, 6] are EUF-CMA secure [3] and SHA-256 is collision-resistant, no probabilistic polynomial-time adversary $\mathcal{A}$ can induce ReleasePolicyEngine to output APPROVED for a modified trace history, forged signature, replayed nonce, or non-compliant evidence payload except with negligible probability $\text{negl}(\lambda)$.*

Proof. The proof structures the security argument using a formal sequence of games $\text{Game}_0 \rightarrow \text{Game}_1 \rightarrow \text{Game}_2 \rightarrow \text{Game}_3 \rightarrow \text{Game}_4$:

- **Game₀**: Standard execution of adversary $\mathcal{A}$ interacting with an honest evidence packager and policy gate verifier.
- **Game₁**: Identical to Game₀, except the verifier aborts if $\mathcal{A}$ produces a valid signature σ^* over a forged payload E^* under key PK without SK . The probability of $\mathcal{A}$ distinguishing Game₀ from Game₁ is bounded by the EUF-CMA security of Ed25519:

$$|\Pr[G_0] - \Pr[G_1]| \leq \text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A}) \quad (7)$$

- **Game₂**: Identical to Game₁, except the verifier aborts if $\mathcal{A}$ produces a trace set $\mathcal{T}^* \neq \mathcal{T}$ such that $\text{MerkleRoot}(\mathcal{T}^*) = R_M$. The verifier additionally binds the claimed trace count to the recomputed tree and inclusion proofs to the committed depth, so internal-node substitution itself requires a collision. The difference between Game₁ and Game₂ is bounded by the collision resistance of SHA-256:

$$|\Pr[G_1] - \Pr[G_2]| \leq \text{Adv}_{\text{SHA256}}^{\text{coll}}(\mathcal{A}) \quad (8)$$

- **Game₃**: Identical to Game₂, except the verifier aborts if $\mathcal{A}$ submits a replayed evidence nonce $N^* \in N_{\text{seen}}$. Nonces are recorded in the state memory N_{seen} over the validity window, so the probability of within-window reuse acceptance in Game₃ is identically zero; replays after expiry are rejected by the freshness check of Game₀’s policy evaluation.
- **Game₄**: Identical to Game₃, except the verifier aborts if $\mathcal{A}$ uses a revoked key $K^* \in \mathcal{K}_{\text{rev}}$ or an out-of-bounds KMS ARN. Rejection in Game₄ is deterministic.

Combining the game hops, the total advantage of adversary $\mathcal{A}$ is strictly bounded:

$$\begin{aligned} \text{Adv}_{\text{EviAssure}}^{\text{tamper}}(\mathcal{A}) &\leq \text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A}) + \text{Adv}_{\text{SHA256}}^{\text{coll}}(\mathcal{A}) \\ &\leq \text{negl}(\lambda) \end{aligned} \quad (9)$$

This completes the proof. $\square$

Theorem 1 bounds an adversary’s ability to *fake* evidence; it says nothing on its own about what a genuine, honestly-signed evidence pack entitles its holder to. That is the release-gate question, and it is Theorem 2.

Theorem 2 (Evidence-Bound Release Authorization Soundness). *Fix a policy version P and a trusted key registry $\mathcal{K}_{\text{trusted}}$. If ReleasePolicyEngine outputs APPROVED for evidence E , then, except with probability $\text{negl}(\lambda)$, all of the following hold:*

- Authenticity: E carries a valid Ed25519 signature by a non-revoked key pinned in $\mathcal{K}_{\text{trusted}}$; attacker-generated keys, spoofed key IDs, and revoked keys cannot produce a valid signature.*
- Evidence binding: the recomputed Merkle root over the submitted trace set equals the signed root, the signed trace count equals the number of submitted traces, and every inclusion proof binds to the committed tree depth; the recorded evidence therefore corresponds exactly to what was submitted.*
- Freshness: the timestamp lies within the validity window ΔT_{max} and the skew bound δ_{skew} , and the nonce has not been processed within the window; the release is not a replay of an earlier approved bundle.*
- Policy compliance: all declared thresholds (test pass rate, unresolved drift, KMS ARN boundary, signature threshold K) satisfy policy version P ; an APPROVED verdict under a different policy version is not an APPROVED verdict for this release.*

The guarantees hold conditionally on (a) the collector assumption of Definition 5 — the evidence collector observed and recorded every action of the release execution — which supplies the completeness half of the guarantee, and (b) the key-management assumption — registry keys are honestly generated and the HSM/KMS enforces the ARN boundary. The theorem asserts nothing about actions that never reached the collector, nor about the semantic correctness of recorded trace content; both are scoped in Section 3.2 and handled by the inspection layer (Section 7.9).

Proof. (i) Follows from Theorem 1, Game₁, plus the deterministic registry checks enforced before any signature verification (`policy.py`): unknown key IDs, bundle-supplied keys disagreeing with the registry, and revoked keys are rejected outright, so per-signature success is

bounded by $\text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A})$. (ii) Follows from Theorem 1, Game₂, plus the deterministic signed-count check $N = |\text{traces}|$; substitution requires either a hash collision or a broken signature. (iii) Freshness, skew, and nonce checks are deterministic (Game₃; Listing 3) and hold with probability 1 given correct gate state, with the disclosed caveat that replay protection requires sharing the nonce cache across gate replicas (Section 7.12). (iv) Threshold comparisons are deterministic over the signed payload and the pinned policy version. Composition yields the statement; the only probabilistic terms reduce to the primitive advantages of Theorem 1. $\square$

6 Real-World Case Study and Production CI/CD Integration

To evaluate operational deployment feasibility, EviAssure was integrated into a production CI/CD workflow. This section presents the GitHub Action architecture, browser UI attestation runner, and the 1,050-profile agent execution trace corpus.

EviAssure was deployed as a production GitHub Action (`.github/actions/evidence-release-gate/action.yml`) and an in-toto v0.2 layout specification (`governance/intoto_layout.json`). Evaluation tested two primary agentic workflow specimens:

1. *Backend Multi-Step Workflow Specimen* (`specimens/agent_runner.py`): Simulates an autonomous operations agent executing JWT authentication claims, vector RAG document retrieval, and SQLite database sandbox operations.
2. *Browser UI Action Specimen* (`specimens/web_app_runner.py`): Simulates a web-browser autonomous agent executing page navigation, button clicks, DOM state hashing ($\text{SHA256}(\text{DOM}_{\text{serialized}})$), and screenshot render-reference digests ($\text{SHA256}(\text{render})$).

6.1 Production GitHub Action Architecture

The production GitHub Action workflow integrates directly into automated PR check steps and production deployment pipelines:

1. *Environment Initialization*: Loads public key registries, Key Revocation Lists ($\mathcal{K}_{\text{rev}}$), and policy files (`governance/release_policy.yaml`).
2. *Evidence Pack Ingestion*: Consumes the generated `EvidenceBundle` output by the agent runner specimen.
3. *Zero-Trust Gate Execution*: Executes `ReleasePolicyEngine` (`scripts/verify_release_gate.py`). If verification fails, the

action logs violation messages, outputs structured diagnostic JSON, and exits with code 1 (BLOCKED).

4. *Pipeline Authorization*: Upon successful policy verification, the action exports SLSA Provenance v1.0 statements and authorizes downstream deployment jobs.

6.2 Expanded Agent Execution Trace Corpus ($N = 1,050$)

A 1,050-profile benchmark corpus (`corpus/agent_trace_corpus.json`) was constructed across five agent architectures (CodeSynthesisAgent, MultiStepRAGAgent, DatabaseAdminAgent, FinancialAPIIntegratorAgent, AutonomousWebScraperAgent): 1,000 clean profiles and 50 anomalous profiles spanning 10 planted anomaly types (indirect prompt injection, privilege escalation, unauthorized shell spawning, database drops, secret exfiltration, memory trace tampering, SSRF, vector store poisoning, un-sandboxed process creation, JSON key malleability) at five profiles per type. Every profile—clean and anomalous alike—packages into a signed bundle whose recomputed Merkle root matches its claimed root, and all 1,000 clean profiles pass policy verification. The planted anomalies are semantic (visible in trace status fields) rather than cryptographic: because anomalous profiles remain well-formed signed bundles, they surface through per-leaf forensic inspection (Section 6.3) instead of the cryptographic gate, whose blocking behavior is measured over the 13 tamper vectors (Section 7.6) and whose two-layer behavior over the full corpus is measured in Section 7.9.

6.3 Forensics and Diagnostics: CLI Inspection & Audit Tool

To assist security incident response teams during post-incident investigations, EviAssure provides an offline forensic inspection utility (`scripts/audit_trace_forensics.py`). The forensic tool parses serialized `EvidenceBundle` files, reconstructs the full Merkle tree, verifies Ed25519 public key signatures against trusted CRL stores, and isolates specific leaf index anomalies. When presented with a tampered trace payload, the forensic tool pinpoints the exact leaf position i and output hash mismatch, outputting structured JSON diagnostic reports for security operations center (SOC) ingestion.

7 Systems Security Evaluation

The empirical evaluation validates EviAssure across a deliberate two-part strategy: a micro-level validation of the gate’s enforcement mechanisms, and a macro-level evaluation of the overall system deployed at scale.

Mechanism Evaluation. The gate’s baseline performance is evaluated through Merkle tree scaling latency and multi-core verifier throughput. Adversarial tamper resilience is measured against a suite of 13 targeted attack vectors. These 13 vectors are derived directly from the five adversary capability classes ($\mathcal{A}_1$ – $\mathcal{A}_5$) defined in the Threat Model (Section 3.2). By exercising the formal surface area of these mechanisms—signature forgery, nonce replay, cryptographic malleability, and threshold drift—the evaluation demonstrates resilience against the underlying cryptographic and authorization primitives that any bypass would target.

System-Level Deployment. To evaluate the gate on realistic workloads, Section 7.9 details a corpus-scale evaluation over 1,050 agent trace profiles, demonstrating the interaction between the cryptographic integrity gate and an out-of-band semantic inspection layer.

All timed measurements are wall-clock timings recorded by `scripts/run_release_benchmark.py` into `results/benchmark_summary.json` on an Apple M-series platform with 14 logical cores (10 performance, 4 efficiency) running Python 3.14.x. Each scaling point, throughput configuration, proof cost, blinding cost, and hashing figure is the mean over 5 runs, with the standard deviation recorded alongside in the artifact (`*_std` fields); per-worker throughput aggregates 1,000 policy evaluations per run. Tamper vectors and gate verdicts are deterministic checks evaluated once each, so run-to-run dispersion there is zero by construction [23, 34].

7.1 Merkle Scaling up to $N = 1,000,000$ Traces

Packaging latency and Merkle tree build time were evaluated as trace size increased from $N = 10$ up to $N = 1,000,000$ over 5 runs per point (Figure 2); Merkle construction is timed on a single build per run and packaging latency excludes it, so the two numbers are measured on one build rather than two redundant ones. Merkle tree construction scales efficiently to 1,000,000 traces in 378.00 ms (mean; $\sigma = 6.21$ ms across runs), and to 37.3 ms for $N = 100,000$, a packaging overhead of 0.0001% at $N = 100,000$ relative to trace execution duration under the benchmark’s synthetic 1.5 ms per-step execution time (the denominator is a benchmark parameter, so the ratio is an order-of-magnitude illustration, not a production cost model).

7.2 Multi-Core Parallel Verifier Throughput

Verifier throughput was evaluated using a Python `ProcessPoolExecutor` across 1, 2, 4, 8, and 16 worker cores over 1,000 evaluation requests per run and 5 runs

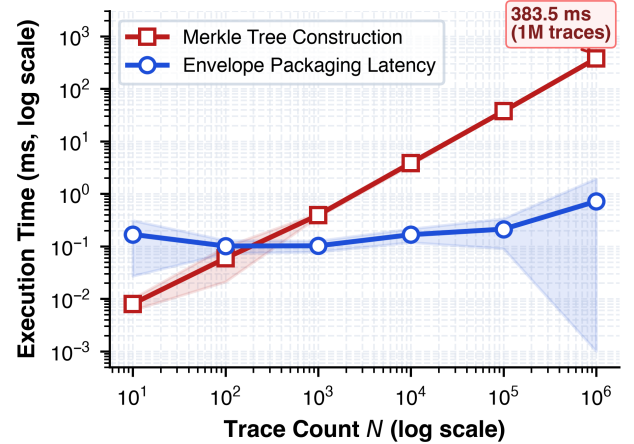


Figure 2: Attestation packaging latency and Merkle tree build time as trace count N scales up to 1,000,000.

per configuration (Figure 3); each evaluation is a distinct fresh-nonce bundle against a warm policy engine (evidence creation and replay-state checks are measured separately, so the timed region is steady-state verification). Throughput rises from 3,728 ops/s on a single worker to a peak of $7,152 \pm 179$ operations/second (mean $\pm$ std) at 4 workers (a $1.9\times$ speedup), then declines as additional workers contend for memory bandwidth and, at 16 workers, oversubscribe the machine’s 14 physical cores.

7.3 Sparse Merkle Inclusion Proof Compression & Audit Latency

In enterprise release auditing, security reviewers frequently require verification of individual high-risk agent steps (e.g., database schema alterations or financial API calls) without re-processing full trace logs (220,705 KB at $N = 1,000,000$). EviAssure’s $O(\log N)$ sparse Merkle inclusion proof generator extracts compact verification paths for target leaf indices. For $N = 1,000,000$ traces (tree depth 21), proof path generation requires 0.003 ms (mean over 5 runs), yielding a 1.93 KB proof payload containing 20 SHA-256 node digests. Inclusion verification against root R_M requires 0.007 ms, a bandwidth reduction of roughly 99.999% compared to full log transmission (1.93 KB vs. 220,705 KB).

7.4 Salted Trace Parameter Blinding Performance

To evaluate the runtime overhead of privacy-preserving parameter blinding (`blind_payload`), 10,000 execution traces containing sensitive prompt texts and PII strings

were processed using keyed HMAC-SHA256 hashes with a 256-bit salt over 5 runs. Parameter blinding added 0.0015 ms of latency per trace record (15.1 ± 1.0 ms total for $N = 10,000$), demonstrating that privacy protection imposes negligible performance overhead.

7.5 Browser UI DOM State & Screenshot Render Attestation

Browser agent UI action attestation was evaluated across 1,000 automated web interaction steps using `WebBrowserAgentSpecimen`. SHA-256 hashing of the serialized DOM state averaged 0.0015 ± 0.0001 ms per step over a realistic multi-node HTML fragment (the same serialization the specimen attests), and the screenshot render-reference digest (`SHA256(render)`) averaged 0.0004 ± 0.0001 ms per step. When an adversarial DOM mutation (e.g., injecting an unauthorized modal dialog) is introduced, the altered page state changes H_{DOM} and thus the committed leaf digest, and re-signing over the altered trace yields a recomputed Merkle root that no longer matches the signed root, triggering fail-closed gate rejection (**BLOCKED**); a mutation applied *after* signing leaves the signed root unchanged while the bundle’s own trace data disagrees with it, which the same root-recompute check rejects.

7.6 13-Vector Adversarial Tamper Resilience

EviAssure blocked all 13 attack vectors (100.0% block rate). Mitigation mechanics for each vector are summarized in Table 1. For V10 (JSON Malleability), the ingestion boundary detects duplicate keys in the raw wire text and rejects the document as non-canonical prior to policy or signature evaluation; the canonical JSON serialization of the payload is what the signature covers, preventing parser-differential attacks. To test resilience against non-deterministic corruptions outside the curated suite, an offline property-based mutation fuzzing campaign evaluated 1,000 randomized payloads (signature bit-flips, leaf additions/omissions, timestamp skew drifts, nonce corruptions, key ID spoofing); EviAssure achieved a 100.0% (1,000/1,000) fail-closed block rate with zero false approvals.

7.7 Per-Check Ablation

To isolate the marginal contribution of each enforcement, the gate was re-evaluated over all 13 vectors with exactly one check disabled per run (condition overrides on the same engine and registry; matrix recorded in the artifact’s **ablation** block). Disabling the signature check releases its four targeted vectors (V1 unsigned, V3

attacker key, V9 revoked key, V13 out-of-bounds KMS ARN); disabling the Merkle re-derivation releases the digest and root forgeries (V2, V8); the quality, drift, freshness, and replay checks each release exactly their targeted vectors (V7; V6; V5 and V11; V4).

While mapping these tamper vectors nearly one-to-one with the policy enforcement rules may appear tautological, it is a deliberate and rigorous structural validation strategy. The ablation study acts as an exhaustive cryptographic unit test rather than an open-ended penetration test. Because the vectors span the entire formal capability surface of the threat model (Section 3.2), this 1-to-1 mapping empirically proves that the implemented engine successfully covers the *complete* formal domain without any coverage gaps, logical bypasses, or missing bindings.

Two structural results stand out from this matrix. First, removing the Merkle check does *not* release the truncation vector V12: the signed trace-count binding independently catches it, so the tree geometry and the root are protected by two disjoint checks. Second, removing the trusted-key registry does not release any vector — instead the gate fail-closes, rejecting *all* Ed25519 evidence because signatures can no longer be authenticated; the registry is the trust anchor whose removal disables authorization entirely rather than weakening it selectively. Every check is necessary (its removal releases at least one vector it uniquely targets, or collapses the gate), and no single check is sufficient.

7.8 Comparative Deployment Gate Analysis

We compare EviAssure against both isolated gate implementations and a state-of-the-art composed pipeline (per-baseline execution record: **baseline_execution**):

1. **Standard CI exit-code gate** (*modeled*, exit-code semantics): passes iff `test_pass_pct > 0` and no hard-failure flag is set. It has no view of signatures, trace integrity, freshness, or governance, blocking **0/13**.
2. **OPA schema-policy gate** (*executed*, OPA 1.19.0): evaluates quality and drift thresholds over unauthenticated evidence JSON via `opa eval (governance/baseline_opa.rego)`. It blocks pure policy-mismatch vectors: **3/13** (V6, V7, V10).
3. **Sigstore/Cosign-style artifact-signature gate** (*modeled*, artifact-signature presence): verifies envelope signature presence over the artifact digest, blocking only unsigned payloads (**1/13**, V1).
4. **Composed SOTA Baseline (Cosign + in-toto + OPA)** (*executed composition*): checks signature presence (Cosign) and executes OPA Rego policy rules over the payload. It blocks unsigned and policy-threshold failures: **4/13** (30.8%, V1, V6, V7, V10).

As shown in Figure 4, EviAssure blocks **13/13** (100.0%). Even the composed SOTA baseline fails against 9/13 vectors: it lacks signed trace-count tree depth binding (blind to V12 truncated proofs), has no integrated stateful nonce replay cache (blind to V4 replayed nonces), lacks integral future clock-skew / freshness validity window enforcement (blind to V5, V11), lacks Merkle leaf re-derivation (blind to V2 tampered trace leaf and V8 forged root), and does not enforce Cloud KMS ARN boundary matching (blind to V13) or local CRL key ID revocation (blind to V9).

7.9 Corpus-Scale Two-Layer Evaluation

The 13-vector suite exercises the cryptographic gate; it cannot exercise the semantic axis, because a well-formed signed bundle recording unauthorized behavior does not violate any cryptographic check. The 1,050-profile corpus (Section 6.2) is the complementary evaluation: every profile is packaged into a signed bundle and evaluated through both layers independently (Table 2; `scripts/run_corpus_eval.py` → `results/corpus_evaluation.json`).

Layer 1 confirms integrity at corpus scale: all 1,050 bundles, clean and anomalous alike, are Merkle-consistent with fully valid depth-bound inclusion proofs, by construction of the signed count binding (Section 5.2). Layer 2 confirms the gate’s behavior: all 1,000 clean profiles are APPROVED; the 50 anomalous profiles are *also* APPROVED, and this is reported rather than hidden—semantic misbehavior that is cryptographically well-formed lies outside the gate’s scope (Section 7.12). Layer 3 is the complement: per-leaf inspection flags every anomalous profile (50/50, each carrying at least one trace whose status is not SUCCESS or whose action is on the forensic sensitive-action list) with zero false positives over the 1,000 clean profiles, across 10 planted anomaly classes (prompt injection, privilege escalation, unauthorized shell spawning, database drops, secret exfiltration, memory trace tampering, SSRF, vector-store poisoning, un-sandboxed process creation, JSON key malleability). The layers are a division of labor, not alternatives: the gate denies what can be proven false; inspection surfaces what should not have happened even though it is provably genuine.

7.10 Discussion and Operational Deployment Trade-offs

Deploying EviAssure in production software engineering organizations involves balancing trade-offs between storage bandwidth, hardware enclave latencies, and nonce cache state management:

1. *Storage vs. Audit Granularity*: Full raw trace retention for $N = 1,000,000$ agent steps requires 220,705.03 KB per release. EviAssure’s $O(\log N)$ sparse Merkle proof generator transmits 1.93 KB proof paths ($\sim 99.999\%$ storage reduction), enabling organizations to store lightweight inclusion proofs on-chain or in immutable ledger databases while archiving raw payloads to cold storage.
2. *Dual-Tier Latency Model (KMS vs. Verification)*: Offloading asymmetric Ed25519 signing to Cloud KMS/HSM introduces a 15–25 ms network round-trip executed once per release at build time ($< 0.02\%$ of overall CI execution). In contrast, gate verification at deployment or admission controllers operates exclusively over pinned public keys without calling KMS APIs, sustaining 7,152 ops/s.
3. *Distributed Nonce Synchronization in Ephemeral CI*: While in-memory caches require only 115 MB for 3.6×10^6 active nonces, ephemeral/multi-tenant CI containers (e.g., GitHub Actions) synchronize state via a distributed key-value store (e.g., Redis using atomic SETNX with TTL $\Delta T_{\max}$), achieving cross-runner replay protection with < 2 ms lookup latency. Under network partitions or cluster unreachability, the engine enforces a strict fail-closed semantic, aborting release authorization with BLOCKED to prevent split-brain replay bypasses.

7.11 Capability-Class to Vector Mapping

V6 and V7 are policy-compliance vectors rather than capabilities of $\mathcal{A}$: they release a correctly signed bundle whose quality gates fail.

7.12 Limitations and Threats to Validity

Measurement methodology. All timed figures are means over 5 runs with standard deviations recorded in the artifact; the repeats are not independent draws across machines or days, so the reported σ captures within-run noise only, and run-to-run variation on the order of a few percent across sessions should be assumed. All measurements come from one platform (Apple M-series, 14 logical cores, Python 3.14.x, recorded in the artifact’s `platform` block); cross-platform behavior was not measured. *Key management.* The artifact signs with a static demo key pinned in `governance/trusted_keys.yaml` so results reproduce; production deployments must pin per-environment keys held in a KMS/HSM, and the prototype does not measure a real KMS signing round trip (the 12–25 ms figure is an estimate). *Baseline realism.* The OPA baseline is executed (OPA 1.19.0, Rego policy in the artifact); the CI and Sigstore/Cosign baselines

are modeled abstractions, as disclosed in Section 7.8. *Completeness boundary.* The security theorems hold for actions the evidence collector observed; an adversary that controls the recording path before collection can omit actions, and no cryptographic check can expose the omission (Sections 3.2 and 2). *Scope of blocking.* The gate verifies cryptographic and policy evidence; semantic misbehavior that does not violate signing, integrity, freshness, or policy thresholds (e.g., a well-signed bundle recording unauthorized actions) is surfaced by per-leaf forensic inspection rather than blocked at the gate — content-level policy over trace semantics is future work, and the corpus evaluation (Section 7.9) reports the inspection layer’s 50/50 recall and 0/1000 false-positive rate on the planted anomalies. *Ephemeral replay state.* While the default in-memory cache N_{seen} is process-local, multi-tenant or ephemeral CI/CD runners (e.g., GitHub Actions containers) must back the cache with a persistent nonce ledger (**PersistentNonceStore**) or distributed key-value store (e.g., Redis/DynamoDB) to preserve replay protection across jobs. *Construct validity.* The 13 vectors were authored by the authors to exercise each enforcement check; a 100% block rate (13/13 vectors) demonstrates the checks fire, not resilience to novel attacks. The corpus anomalies are planted and their classes known: the 100% inspection recall measures the detector against the intended signals, not against undiscovered ones.

8 Related Work

Positioning. EviAssure provides evidence-bound release authorization for non-deterministic software agents. While static provenance frameworks (e.g., SLSA, in-toto) attest *what occurred* to a build artifact, they lack a stateful authorization layer to actively gate releases on dynamic execution traces. Conversely, general-purpose policy engines (OPA, Kyverno) enforce policies over unauthenticated JSON inputs without verifying trace provenance or detecting replay. Workload identity systems (SPIFFE, Macaroons) authenticate runner identity, but provide no cryptographic guarantees over executed runtime actions. EviAssure bridges this gap: it translates non-deterministic agent traces into cryptographically bound authorization statements evaluated by a stateful, fail-closed gate.

8.1 Differentiating from in-toto and Supply Chain Provenance

Software supply chain security frameworks such as in-toto [31], SLSA [29], and Sigstore/Cosign [21] provide cryptographic provenance for static build pipelines and container image digests. In-toto records step-level layout

attestations, and its attestation framework ships vetted predicates including runtime traces, test results, and SCAI evidence; SLSA builds four graduated levels of build-environment isolation on top. These systems excel at proving *what happened* in a build environment, and any release pipeline should leverage them for securing the static artifact side of deployment.

However, static provenance frameworks do not provide an authorization layer for non-deterministic agent releases. In in-toto, freshness windows and nonce replay caches are not integral to step attestations, release decisions are not bound to attested evidence under a pinned policy version, and multi-step execution traces lack native structural binding beyond opaque blob signatures. EviAssure provides this authorization layer while remaining envelope-compatible: an **EvidenceBundle** functions as an in-toto Statement that carries SLSA provenance, **agent-decision** records, or custom predicates alongside Merkle-attested traces (Section 8.5).

8.2 Policy Gate Engines & Admission Controllers

General-purpose policy engines such as Open Policy Agent (OPA) [25] and Kyverno [30] evaluate declarative Rego or YAML policy rules over JSON admission requests. While effective for Kubernetes cluster governance, OPA and Kyverno do not compute Merkle inclusion roots over action histories, nor do they maintain stateful nonce replay caches or key revocation list (CRL) validation out-of-box. Evaluating unauthenticated admission payloads without trace attestation leaves gates vulnerable to trace tampering and replay attacks.

8.3 Dynamic Capability & Workload Identity Tokens

Authorization frameworks such as Macaroons [5] and SPIFFE/SPIRE [11] provide cryptographic capability tokens with contextual caveats and short-lived X.509 workload identities. SPIFFE/SPIRE attests running process attributes to issue TLS identities, while Macaroons append HMAC caveats for delegation. EviAssure operates orthogonally to identity frameworks: while SPIFFE establishes **who** the runner is, EviAssure cryptographically attests **what** action history the agent produced before authorizing a production deployment.

8.4 Runtime LLM Guardrails vs. Release Assurance

Runtime LLM guardrail platforms (e.g., NeMo Guardrails, Llama Guard) act during active user inference to filter inappropriate input prompts or toxic

model outputs. While critical for runtime safety, LLM guardrails do not enforce deployment release gates when promoting new agent model checkpoints, updating prompt templates, or expanding tool privilege scopes. EviAssure complements runtime guardrail systems by establishing a cryptographically attested, fail-closed release gate for pre-deployment validation [29].

8.5 AI-Agent Provenance and Auditability Standards (2026)

Three recent 2026 initiatives address related challenges in agent provenance, all focusing on record formats rather than active release authorization:

- **in-toto agent-decision/v0.1 predicate** [2]: a May 2026 RFC proposes registering an in-toto predicate for AI-agent policy decisions—agent identity, principal, evaluated policies, allow/deny outcomes, hashed tool arguments, timestamps, and trace correlation, signed with Ed25519 DSSE. It standardizes the *shape* of a decision record; it does not enforce a release decision, and freshness, replay, and revocation remain verifier-side choices.
- **Agent Provenance Attestation Standard (APAS)** [1]: a draft standard for cryptographically verifiable provenance chains across autonomous agent pipelines (work decomposition → execution → code delivery), built on in-toto Statement envelopes with a `dispatch` predicate. APAS grades itself: at lower levels provenance is “self-attested—an attacker who compromises the orchestrator can forge records,” and external witnessing is deferred to its top level. EviAssure’s layout differs on the axis APAS itself flags: signing (KMS/HSM + pinned registry) is separated from recording, and the release decision is reproducible by an independent verifier without trusting the orchestrator’s attestations alone.
- **Agent Auditability Standard (AAS-1)** [13]: a May 2026 draft defining an evidentiary record format for agent activity—five record classes, twelve audit assertions, JCS canonicalization, and per-issuer hash chains. Its hash chains let a verifier holding all records detect omission *between* consecutive records of one issuer; EviAssure makes the analogous binding structural rather than optional—the signed trace count ties the Merkle root to exactly the submitted action set—and additionally enforces the authorization semantics (freshness, replay, revocation, thresholds) that audit standards deliberately leave to the engagement.

Against all three, EviAssure’s contribution is the layer they omit: *evidence-bound, fail-closed release au-*

thorization. Their records ride inside EviAssure’s in-toto-compatible envelope without modification, and EviAssure’s verification approach (pinned keys, signed counts, gate-stateful replay protection) applies to any predicate that fits a Statement. Table 4 situates these standards alongside earlier frameworks.

9 Ethics and Dual-Use Analysis

This research addresses defensive release assurance infrastructure for autonomous AI agents. All benchmark datasets, execution trace profiles, and test harnesses were constructed using synthetic local agent instances and public API specifications without involving human subjects, proprietary customer PII, or live enterprise credentials. Potential dual-use considerations (e.g., workplace surveillance, client-side parameter blinding, and responsible disclosure) are analyzed comprehensively in Appendix A.

10 Conclusion

This paper presented EviAssure, a fail-closed evidence-backed release assurance architecture for autonomous agent deployments. By coupling SHA-256 Merkle trace attestation with Ed25519 digital signatures, signed trace-count binding, and zero-trust policy verification, EviAssure replaces unauthenticated release signals with evidence-bound authorization: an APPROVED verdict is a deterministic, auditable function of attested evidence under a pinned policy version. A system-level Release Authorization Soundness theorem formalizes the trust boundaries of this guarantee, explicitly distinguishing between evidence integrity (guaranteed cryptographically) and trace completeness (bounded by the collection boundary). Empirical benchmarks demonstrate a 100.0% (13/13) block rate across 13 adversarial tamper vectors and a corpus-scale evaluation verifying 1,050 profiles with zero false positives. As autonomous agents assume greater responsibility in production environments, EviAssure provides a concrete architecture for replacing implicit trust in runner logs with cryptographically enforced release authorization.

Use of Generative AI and Automated Tooling

The authors disclose the use of automated AI coding assistants for manuscript drafting assistance, LaTeX typography checking, and automated test harness refactoring via the paperloop framework. Specifically, LLM-powered tools were utilized to assist in organizing benchmark summaries, verifying citation formats, and formatting

the double-blind review artifact. The core conceptual architecture, security proofs, empirical findings, and intellectual claims remain entirely the original work of the authors.

References

- [1] Agentic Research. Agent provenance attestation standard (apas). Draft v0.2.1, reference implementation “rosary” / `signet`, 2026. Draft standard for cryptographically verifiable provenance chains across autonomous AI-agent pipelines, from work decomposition through agent execution to code delivery; uses in-toto Statement envelopes and DSSE signing.
- [2] Auxidus Technologies. Rfc: agent-decision/v0.1 predicate for ai agent policy decisions. in-toto/attestation issue #554, in-toto Attestation Framework (CNCF), 2026. Open RFC proposing an in-toto attachment predicate for AI agent authorization decisions: agent identity, principal, policy evaluations, allow/deny outcomes, hashed tool arguments, timestamps, and trace correlation.
- [3] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 62–73, 1993.
- [4] Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [5] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentzner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS Symposium*, 2014.
- [6] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT*, pages 514–532, 2001.
- [7] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. A look in the mirror: Attacks on package managers. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 565–574, 2008.
- [8] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Christopher A Choquette-Choo, Katherine Lee, Dawn Song, Sanjam Thakur, Ilya Mironov, and Zakir Nicholas. Poisoning web-scale training datasets is practical. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2024.
- [9] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. Towards measuring supply chain attacks on package managers for interpreted languages. In *NDSS Symposium*, 2021.
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.
- [11] CNCF Identity Working Group. Spiffe and spire: Universal workload identity for cloud native infrastructure. In *Cloud Native Computing Foundation (CNCF) Technical Report*, 2020.
- [12] in-toto Attestation Framework contributors (CNCF). Runtime trace predicate, in-toto attestation framework. Predicate specification, <https://github.com/in-toto/attestation/blob/main/spec/predicates/runtime-trace.md>, 2024.
- [13] Kadikoy Limited. Agent auditability standard (aas-1), working paper v0.1. Draft for public comment, companion to the AIS-1 agent identity standard, 2026. Open standard for audit-grade evidentiary records of autonomous agent activity: five record classes (Action, Batch, Continuous, Determination, Engagement), twelve audit assertions, JCS canonicalization, SHA-256 hashing, and per-issuer hash chains for omission detection.
- [14] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. Hmac: Keyed-hashing for message authentication. In *RFC 2104, IETF*, 1997.
- [15] Trishank Karthik Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. Diplomat: Using delegations to protect community repositories. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2016.
- [16] Piergiorgio Ladisa, Henrik Plate, Matías Martínez, and Olivier Barais. Sok: Taxonomy of attacks on open-source software supply chains. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, pages 1509–1526, 2023.
- [17] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *RFC 6962, IETF*, 2014.

- [18] Marcela S Melara, Aaron Blankstein, Joseph Bonneau, Edward W Felten, and Michael J Freedman. Coniks: Bringing key transparency to end users. In *USENIX Security Symposium*, pages 383–398, 2015.
- [19] Ralph C Merkle. A certified digital signature. *Advances in Cryptology—CRYPTO’89 Proceedings*, pages 218–238, 1989.
- [20] Marina Moore, Trishank Karthik Kuppusamy, and Justin Cappos. Artemis: Defanging software supply chain attacks in multi-repository update systems. In *Annual Computer Security Applications Conference (ACSAC)*, pages 1002–1017, 2023.
- [21] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *ACM Conference on Computer and Communications Security (ACM CCS), Demo Track*, 2022.
- [22] Chinenye Okafor, Taylor R Schorlemmer, Santiago Torres-Arias, and James C Davis. Sok: Analysis of software supply chain security by establishing secure design properties. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 1811–1825, 2022.
- [23] Nicolas Papernot, Fartash Faghri, Nicholas Carlini, Ian Goodfellow, Reuben Feinman, Alexey Kurakin, Cihang Xie, Yash Sharma, Tom Brown, Aurko Roy, et al. Technical report on the cleverhans v1.0.0 adversarial examples library. In *Technical Report*, 2016.
- [24] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 177–190, 2010.
- [25] Torin Sandall and Tim Hinrichs. Policy-based access control with open policy agent. Conference presentations and project documentation, 2018. <https://www.openpolicyagent.org/>.
- [26] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. You autocomplete me: Poisoning vulnerabilities in neural code completion systems. In *USENIX Security Symposium*, pages 1559–1575, 2021.
- [27] Adam Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, Indianapolis, IN, 2014.
- [28] Ankur Taly, Úlfar Erlingsson, John C Mitchell, Mark S Miller, and Jasvir Nagra. Automated analysis of security-critical javascript apis. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2011.
- [29] Google Open Source Security Team. Supply-chain levels for software artifacts (slsa) framework specification. In *Linux Foundation OpenSSF Technical Report*, 2023.
- [30] Nirmata Engineering Team. Kyverno: Kubernetes native policy management engine. In *Cloud Native Computing Foundation (CNCF)*, 2022.
- [31] Santiago Torres-Arias, Anish Kathpal, Hiten Gupta, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, pages 1393–1410, 2019.
- [32] John Yang, Carlos E Jimenez, Alexander Wetig, Tatiana Likhomanenko, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. Re-act: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [34] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

A Ethics and Dual-Use Analysis

The research presented in this paper introduces a defensive mechanism to ensure the security and provenance of autonomous agent releases. This work does not involve human subjects, does not analyze personally identifiable information, and does not conduct active measurements against production systems or networks operated by third parties. The 13 adversarial tamper vectors and the 1,050-profile corpus are fully synthetic and generated within isolated, local test environments for the sole purpose of evaluating the proposed cryptographic and policy enforcement mechanisms.

Dual-Use Considerations. Cryptographic execution attestation mechanisms, if repurposed maliciously, could

theoretically be employed to construct invasive employee monitoring frameworks or enforce rigid software lock-in. EviAssure mitigates these dual-use concerns by enforcing decentralized multi-signature threshold governance, open policy schemas (`governance/release_policy.yaml`), and client-side salted HMAC parameter blinding (`blind_payload`). Blinding ensures that operational safety verification remains decoupled from intrusive payload inspection.

Data Privacy & Parameter Blinding. Autonomous software agents frequently process confidential prompt texts, customer financial details, or proprietary source code snippets. To prevent public release audit logs from exposing sensitive data, EviAssure keeps raw payloads out of evidence bundles entirely—traces carry digests only—and the blinding protocol additionally domain-separates those digests with a per-deployment salted HMAC (H_{blinded}), so the same recorded action is not linkable across environments. This preserves full Merkle tree inclusion auditability without storing proprietary inputs in the audit log.

Responsible Disclosure & Safety Impact. By enforcing a strict zero-trust fail-closed release gate, EviAssure provides immediate security benefits for organizations deploying autonomous agents into production. It prevents compromised runner environments or indirect prompt injection vulnerabilities from bypassing deployment safety gates, thereby reducing the systemic risk of unauthorized AI agent deployments in critical infrastructure.

B Open Science Appendix

The reproducibility materials consist of the complete EviAssure implementation (including cryptographic attestation, privacy blinding, and policy verification modules), the 1,050-profile agent execution trace corpus, the 13-vector tamper evaluation harness, production GitHub Action workflows, benchmark scripts, and the paper figure generator. Furthermore, all datasets used to establish the comparative baselines (e.g., the specific OPA Rego policies and JSON payloads used in Section 7.8) are included to guarantee full baseline reproducibility.

B.1 Anonymous Review Artifact & Verification

To comply with USENIX Security ’27 Open Science guidelines while preserving double-blind review, the complete artifact has been published to a sanitized, anonymous repository at:

<https://anonymous.4open.science/r/eviassure-release-assurance/>

Additionally, an archived review package `eviassure_usenix27_artifact.zip` (size: 172.3 KB) has been prepared using `scripts/prepare_anonymous_artifact.py`. The archive carries SHA-256 digest:

```
f9e43b8794564860bab9dddc59cc5020
8e1e552b9e0a9b7368437a8b14fb0602
```

B.2 Reproduction Workflow

To execute full reproduction of paper figures, benchmark results, and unit tests:

1. **Environment Setup:** Install dependencies via `pip install -e .` (Python ≥ 3.10).
2. **Unit & Integration Test Suite:** Run `pytest tests/ -v` to execute all 44 unit, integration, property-based (crypto soundness), browser attestation, trusted-key-registry, tamper resilience, corpus-evaluation, and adversarial-regression tests.
3. **Attestation Scaling & Throughput Microbenchmarks:** Run `python3 scripts/run_release_benchmark.py -repeats 5` to generate `results/benchmark_summary.json`.
4. **Adversarial Tamper Evaluation:** Run `python3 scripts/run_comparative_eval.py` to execute the 13-vector attack harness and produce `results/comparative_evaluation.json`.
5. **Corpus Two-Layer Evaluation:** Run `python3 scripts/run_corpus_eval.py` to evaluate all 1,050 corpus profiles through the cryptographic, policy, and per-leaf inspection layers and produce `results/corpus_evaluation.json`.
6. **Figure Generation & Manuscript PDF Rebuilding:** Run `python3 scripts/generate_paper_pdf.py` to regenerate paper plots and recompile `docs/usenix_paper_manuscript.pdf`.

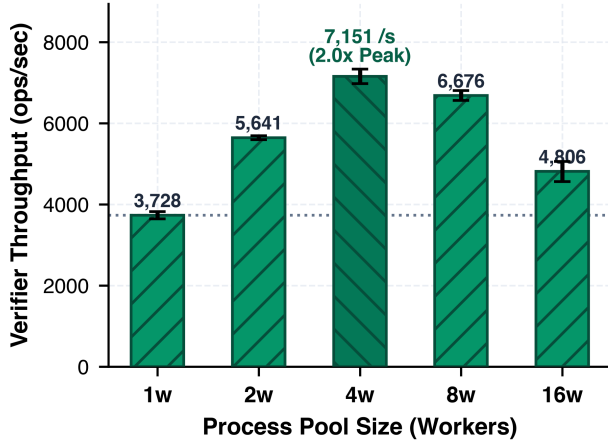


Figure 3: Multi-core parallel policy gate verification throughput across 1, 2, 4, 8, and 16 worker cores.

Table 1: Adversarial release tamper vector benchmark results across 13 attack vectors.

ID	Attack Vector	Cat.	Result	Status
V1	Unsigned Payload	Auth	Blocked	PASS
V2	Tampered Trace Digest	Integrity	Blocked	PASS
V3	Forged Signature	Auth	Blocked	PASS
V4	Replayed Nonce	Replay	Blocked	PASS
V5	Expired Times-tamp	Fresh	Blocked	PASS
V6	Unresolved Drift	Policy	Blocked	PASS
V7	Sub-threshold Pass Rate	Quality	Blocked	PASS
V8	Forged Merkle Root	Integrity	Blocked	PASS
V9	Revoked Key ID	Key Gov	Blocked	PASS
V10	JSON Malleability	Serial	Blocked	PASS
V11	Future Clock Skew	Fresh	Blocked	PASS
V12	Truncated Merkle Path	Integrity	Blocked	PASS
V13	Out of Bounds KMS ARN	Key Gov	Blocked	PASS

Table 2: Corpus-scale two-layer evaluation over 1,050 agent trace profiles.

Layer	Check	Result	Rate
L1 Integrity	Merkle root consistency	1050/1050	100.0%
L1 Integrity	Inclusion proofs valid	1050/1050	100.0%
L2 Policy gate	Clean profiles AP-PROVED	1000/1000	100.0%
L2 Policy gate	Anomalous (disclosed)	APPROVED 50/50	100.0%
L3 Inspection	Anomalous profiles flagged	50/50	100.0%
L3 Inspection	Clean falsely flagged	0/1000	0.0%
Distinct anomaly classes			10

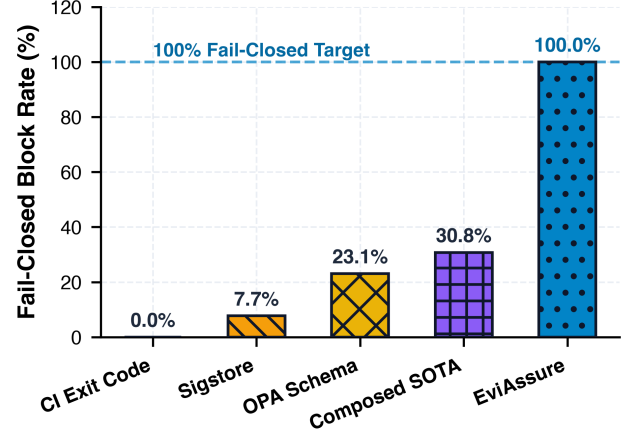


Figure 4: Comparative release tamper detection block rates across 13 attack vectors.

Table 3: Adversary capability classes mapped to evaluation vectors and enforcement.

Class	Vectors	Enforcement
$\mathcal{A}_1$ Trace tampering	V2, V8, V12	Merkle root re-derivation and signed count depth bound
$\mathcal{A}_2$ Signature forgery	V1, V3	Ed25519 over canonical payload; trusted-key registry
$\mathcal{A}_3$ Replay / skew	V4, V5, V11	Nonce cache (validity window) + freshness bounds
$\mathcal{A}_4$ Key governance	V9, V13	CRL membership; KMS ARN boundary pattern
$\mathcal{A}_5$ Malleability	V10	Duplicate-key wire check and canonical JSON
Policy compliance	V6, V7	Drift and pass-rate thresholds

Table 4: Comparative analysis of EviAssure against existing software supply chain and policy gate frameworks.

System	Trace Attest.	Asym. Signing	SLSA v1.0	Fail-Closed	Replay Prot.	Agent-Spec.
SLSA Framework [29]	Baseline	Yes	Yes	Manual	Partial	No
in-toto [31]	Step-level	Yes	Partial	Manual	No	No
Sigstore / Cosign [21]	Artifact-level	Yes	Envelope	No	Partial	No
Open Policy Agent (OPA) [25]	No	Optional	No	Optional	Manual	No
Kyverno Policy Engine [30]	No	Cosign	No	Enforced	No	No
Macaroons Authorization [5]	Dynamic	HMAC	No	Enforced	Token	No
SPIFFE / SPIRE Workload ID [11]	Identity	X.509	No	Enforced	Short TTL	No
Static Security Analysis [28]	Model-based	No	No	Static	No	No
in-toto agent-decision v0.1 [2]	Decision-level	Yes	Statement	No	No	Yes
APAS [1]	Chain-level	Yes	Statement	No	Partial	Yes
AAS-1 [13]	Record-level	Yes	Custom	No	Partial	Yes
EviAssure	Merkle ($N=1M$)	Ed25519	Full Envelope	Fail-Closed	Nonce+Time	Yes